

DBMS LABORATORY EXAM PREPARATION GUIDE

Comprehensive Step-by-Step Practical & Theoretical Walkthrough

This document provides an exhaustive, structured guide for the upcoming Database Management Systems (DBMS) Laboratory Exam. It covers all key practical tasks, syntax structures, core theoretical mechanisms, and interface workarounds documented across all three lab manuals.

Module 1: DBMS Transactions & Transaction Control Language (TCL)

A database transaction is a single logical unit of data processing that executes one or more operations (such as reads, inserts, updates, or deletes) as an indivisible package. Transactions protect data integrity by adhering strictly to an "all-or-nothing" paradigm.

 **CRITICAL INTERFACE NOTE:** phpMyAdmin utilizes an automated client-side JavaScript linter that highlights commands like `SAVEPOINT` or `ROLLBACK TO SAVEPOINT` as errors (Ⓢ) before execution. These are false positives. Disregard the visual markers, select the complete block, and click "Go" to execute successfully.

Step 1: Environment Setup & Session Configuration

By default, MySQL automatically saves every state alteration instantly. To execute and test multi-step transaction controls, this feature must be explicitly disabled for your active session, and tables must utilize a transactional engine (InnoDB).

```
-- Disable automated commit processing for the active session
SET autocommit = 0;

-- Create a dedicated lab database
CREATE DATABASE LabTransactions;
USE LabTransactions;

-- Initialize the accounts table with the InnoDB Storage Engine
CREATE TABLE Accounts (
    account_id INT PRIMARY KEY,
    name VARCHAR(50),
    balance DECIMAL(10,2)
) ENGINE=InnoDB;
```

Step 2: Seed Starting Test Data

```
INSERT INTO Accounts VALUES (1, 'Alice', 1000.00);
INSERT INTO Accounts VALUES (2, 'Bob', 500.00);
```

Step 3: Exercise 1 — Successful Transaction Lifecycle (COMMIT)

Simulates a complete, error-free funds transfer of \$200.00 from Alice to Bob. Both steps must succeed for changes to be permanently written to non-volatile disk storage.

```
START TRANSACTION;

-- 1. Deduct funds from Alice's account balance
UPDATE Accounts
SET balance = balance - 200.00
WHERE account_id = 1;

-- 2. Add matching funds to Bob's account balance
UPDATE Accounts
SET balance = balance + 200.00
WHERE account_id = 2;

-- 3. Finalize and permanently save the state modifications
COMMIT;
```

Verification: Execute `SELECT * FROM Accounts;`. Expected Result: Alice has 800.00, Bob has 700.00.

Step 4: Exercise 2 — Failed Transaction State Recovery (ROLLBACK)

Simulates a data processing anomaly or systemic failure midway through a \$300.00 transfer. The system must erase any uncommitted modifications and revert to its exact pre-transaction boundary.

```
START TRANSACTION;

-- 1. Deduct partial funds from Alice
UPDATE Accounts
SET balance = balance - 300.00
WHERE account_id = 1;

-- [SIMULATED PROCESSING ERROR OR HARDWARE CRASH OCCURS HERE]
-- The database administrator or application triggers an explicit rollback

ROLLBACK;
```

Verification: Execute `SELECT * FROM Accounts;`. Expected Result: Alice remains at 800.00, Bob remains at 700.00. The partial deduction is safely wiped from temporary memory cache.

Step 5: Exercise 3 — Partial Rollbacks & Landmarks (SAVEPOINT)

Savepoints allow the separation of a single transaction block into sub-components, enabling selective recovery of specific mistaken commands while preserving valid updates.

```

START TRANSACTION;

-- 1. Apply a valid corporate bonus reward to Alice
UPDATE Accounts
SET balance = balance + 100.00
WHERE account_id = 1;

-- 2. Establish a landmark checkpoint boundary
SAVEPOINT BonusAdded;

-- 3. Apply an accidental, mistaken corporate bonus to Bob
UPDATE Accounts
SET balance = balance + 500.00
WHERE account_id = 2;

-- 4. Purge ONLY the processing mistake targeting Bob's account
ROLLBACK TO SAVEPOINT BonusAdded;

-- 5. Commit remaining valid changes to the database engine
COMMIT;

```

Verification: Execute `SELECT * FROM Accounts;`. Expected Result: Alice successfully secures her bonus (900.00). Bob's accidental allocation is discarded, keeping his balance at 700.00.

Module 2: Relational Database Normalization (1NF → 2NF → 3NF)

Normalization optimizes schema architecture by breaking down bloated tables to mitigate redundancy and permanently protect against data anomalies:

- **Update Anomaly:** Redundant multi-location data copies mean modifying a single attribute requires sweeping row updates. Missing a row creates database contradictions.
- **Insertion Anomaly:** Structured properties prevent recording new valid facts because unrelated field values are missing.
- **Deletion Anomaly:** Erasing an independent record unintentionally purges distinct, critically independent operational historical data.

Stage 1: First Normal Form (1NF) — Atomic Values Only

The Core Mandate: Every single cell column matrix intersection must contain a single, indivisible *atomic* value. Lists, multi-value fields, or arrays hiding inside a column are strictly prohibited.

Decomposition Execution: Comma-separated listings are systematically expanded into separate rows. A multi-column **Composite Primary Key** is defined to ensure absolute structural row uniqueness.

Stage 2: Second Normal Form (2NF) — Eliminate Partial Dependencies

The Core Mandate: The schema must satisfy 1NF, and every non-key attribute must depend on the *entirety* of the primary key. It applies directly to tables with composite primary keys.

Decomposition Execution: Attributes determined by only a fraction of a composite primary key are extracted into a new child table, mapped back using referential hooks.

Stage 3: Third Normal Form (3NF) — Eliminate Transitive Dependencies

The Core Mandate: The schema must satisfy 2NF, and no non-key attribute can rely on another non-key attribute. All columns must depend solely and directly on the primary key.

Decomposition Execution: Indirect dependency chains are systematically split into independent lookup tables, preserving connections with explicit Foreign Keys.

The Complete, Fully Normalized 3NF Production Script

```
-- 1. Initialize Primary Entity for Students
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(100)
);

-- 2. Isolate Instructor Attributes to Resolve Transitive 3NF Violations
CREATE TABLE Instructors (
    Instructor VARCHAR(100) PRIMARY KEY,
    Department VARCHAR(100)
);

-- 3. Construct Courses Table Referencing Parent Instructors Table
CREATE TABLE Courses (
    CourseName VARCHAR(100) PRIMARY KEY,
    Instructor VARCHAR(100),
    FOREIGN KEY (Instructor) REFERENCES Instructors(Instructor)
);

-- 4. Form the Composite Associative Table to Resolve 2NF Many-to-Many Mappings
CREATE TABLE Enrollments (
    StudentID INT,
    CourseName VARCHAR(100),
    PRIMARY KEY (StudentID, CourseName),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseName) REFERENCES Courses(CourseName)
);
```

Business Report Reconstruction via Multilateral JOINS

```
SELECT S.StudentID, S.StudentName, E.CourseName, C.Instructor, I.Department
FROM Students S
INNER JOIN Enrollments E ON S.StudentID = E.StudentID
INNER JOIN Courses C ON E.CourseName = C.CourseName
INNER JOIN Instructors I ON C.Instructor = I.Instructor;
```

Module 3: Integrity Constraints & Index Optimization

Task 1: Evaluating Referential Action Behaviors (ON DELETE)

Foreign keys act as database guardians, preserving relational consistency across tables through explicit behavioral constraints.

```

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE
);

CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(150) NOT NULL,
  price DECIMAL(10, 2) NOT NULL
);

CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  user_id INT,
  product_id INT,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE RESTRICT
);

```

Practical Referential Rule Verifications

- **Testing ON DELETE RESTRICT:** Running `DELETE FROM products WHERE id = 1;` while an order references that product fails with error `#1451`. MySQL blocks the deletion to preserve transaction history.
- **Testing ON DELETE CASCADE:** Running `DELETE FROM users WHERE id = 1;` succeeds. MySQL automatically purges all child order rows mapped to user 1, preventing orphaned data.

Task 2: Query Performance Diagnostics via EXPLAIN

Indices establish direct data pathways, allowing MySQL to locate records instantly without reading the entire physical disk layout.

```

-- Setup Mock Inventory Structure
CREATE TABLE inventory (
  id INT AUTO_INCREMENT PRIMARY KEY,
  sku VARCHAR(50) NOT NULL UNIQUE,
  product_name VARCHAR(150) NOT NULL,
  category_unindexed VARCHAR(100) NOT NULL,
  stock_count INT NOT NULL,
  INDEX idx_product_name (product_name)
);

```

Analyzing the Execution Profile Metrics

```

EXPLAIN SELECT * FROM inventory WHERE category_unindexed = 'Computers';
EXPLAIN SELECT * FROM inventory WHERE product_name = 'Apple MacBook Pro 16';

```

Execution Query Command	Diagnostic Type	Possible Keys Used	Rows Searched	Engine Operational Behavior
ALL	NULL	6 (All rows)	Full Table Scan: MySQL physically inspects every row on the disk (highly inefficient).	
ref	idx_product_name	1	Index Lookup: Bypasses table scans, using the index tree to jump straight to the exact row.	

Multi-Column Index Optimization (Composite Indexing)

When query filters span multiple parameters in a **WHERE** clause, individual separate indexes are sub-optimal. A composite index satisfies both rules in a single step.

```
-- Establish a multi-column compound optimization pathway
ALTER TABLE inventory ADD INDEX idx_name_stock (product_name, stock_count);

-- Diagnose optimization efficiency
EXPLAIN SELECT * FROM inventory WHERE product_name = 'Razer DeathAdder Gaming Mouse' AND
stock_count > 0;
```

Why it works: MySQL utilizes the compound **idx_name_stock** map path to filter both the product name and stock level simultaneously during a single index traversal pass, completely avoiding row filtering in secondary memory blocks.